

Du Bagging au Random Forest : Fondements Mathématiques et Importance des Variables

Yanis Remmache¹¹Ingénieur Chercheur Data Scientist

Publié le 12 avril 2026

Résumé—Ce cours explore la transition théorique et pratique du bagging vers les forêts aléatoires (Random Forest). Nous détaillons comment l'agrégation de modèles permet de réduire la variance via la décomposition biais-variance, l'utilité de l'erreur Out-Of-Bag pour la validation, et les mécanismes d'évaluation de l'importance des variables.

Keywords—Random Forest, Bagging, Feature Importance, Out-of-bag error, Scikit-learn

1. INTRODUCTION

L'algorithme Random Forest est une méthode d'apprentissage supervisé ensembliste combinant le bagging et les arbres de décision (souvent CART). Ce paradigme vise principalement à réduire la variance des prédictions sans dégrader le biais. Pour le démontrer, nous nous appuyons sur la décomposition biais-variance de l'erreur quadratique moyenne (MSE). Soit le modèle théorique :

$$Y = f(\mathbf{x}) + \epsilon$$

Où Y est la cible, f la "vraie" fonction déterministe, et ϵ un bruit aléatoire de moyenne nulle et de variance σ^2 . La fonction f est estimée à partir d'un échantillon d'apprentissage $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)\}$. On note $\hat{f} := \hat{f}(\mathcal{D}_n)$ le prédicteur associé :

$$\hat{Y} = \hat{f}(\mathbf{x})$$

Au point $\mathbf{x}$, l'erreur quadratique moyenne se décompose ainsi :

$$\mathbb{E}((Y - \hat{Y})^2) = (\mathbb{E}(\hat{f}(\mathbf{x})) - f(\mathbf{x}))^2 + V(\hat{f}(\mathbf{x})) + \sigma^2 \quad (1)$$

$$= \text{Biais}^2(\hat{f}(\mathbf{x})) + V(\hat{f}(\mathbf{x})) + \sigma^2 \quad (2)$$

Pour bien comprendre la nature de chaque terme, il est crucial de considérer l'estimateur $\hat{f}$ comme une **fonction aléatoire** dépendant de l'échantillon d'entraînement : $\hat{f}(\mathbf{x}) := \hat{f}(\mathcal{D}_n; \mathbf{x})$.

- **Le Biais** représente l'erreur moyenne commise par $\hat{f}$ au point $\mathbf{x}$, sur tous les échantillons possibles $\mathcal{D}_n$. Un biais élevé indique que systématiquement, le modèle ne parvient pas à capturer la structure réelle de $f(\mathbf{x})$, signe d'un *sous-apprentissage* (modèle trop simple).
- **La Variance** mesure la dispersion de la prédiction $\hat{f}(\mathbf{x})$ autour de sa propre valeur moyenne $\mathbb{E}[\hat{f}(\mathbf{x})]$. Une variance élevée signifie que la fonction apprise est extrêmement sensible aux fluctuations spécifiques de l'échantillon $\mathcal{D}_n$, ce qui caractérise un *sur-apprentissage* (overfitting).
- **Le Bruit** (σ^2) est la variance de l'erreur irréductible ϵ . Il représente la limite théorique de l'erreur que l'on ne peut éliminer, quelles que soient les performances du modèle.

2. LE BAGGING

2.1. Principe du bagging

Le bagging (*bootstrap aggregating*) est une méthode conçue pour améliorer la stabilité des algorithmes d'apprentissage. Au lieu de s'appuyer sur un seul modèle, on agrège B modèles indépendants. Chaque modèle est entraîné sur un échantillon bootstrap $\mathcal{D}_b$ (tirage avec remise de n individus parmi n).

En régression, la prédiction finale $\hat{f}_{bag}$ est la moyenne des prédictions. En classification, on utilise le vote à la majorité : la classe

prédite est celle qui recueille le plus de suffrages parmi les B modèles. L'algorithme 1 résume ce processus.

Algorithm 1 Algorithme de Bagging

Require: Échantillon $\mathcal{D}_n$, nombre de modèles B .

```

1: for  $b = 1$  to  $B$  do
2:   Générer un échantillon bootstrap  $\mathcal{D}_b$  par tirage avec remise dans  $\mathcal{D}_n$ .
3:   Entraîner un modèle  $\hat{f}_b$  sur  $\mathcal{D}_b$ .
4: end for
5: Agrégation :
6: if Régression then
7:    $\hat{f}_{bag}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})$ 
8: else if Classification then
9:    $\hat{f}_{bag}(\mathbf{x}) = \text{mode}\{\hat{f}_1(\mathbf{x}), \dots, \hat{f}_B(\mathbf{x})\}$ 
10: end if
11: return  $\hat{f}_{bag}$ 

```

2.2. Justification théorique : réduction de la variance

L'intérêt du bagging est de réduire la variance sans augmenter le biais. Considérons une classe d'algorithmes $\mathcal{F}$. D'un côté, nous estimons $\hat{f} \in \mathcal{F}$ sur l'ensemble de l'échantillon $\mathcal{D}_n$. De l'autre, nous construisons B prédicteurs $\hat{f}_1, \dots, \hat{f}_B \in \mathcal{F}$ sur les sous-échantillons bootstrap $\mathcal{D}_1, \dots, \mathcal{D}_B$. Puisque chaque prédicteur est issu de $\mathcal{F}$, chacun partage la même espérance et la même variance : $\mathbb{E}(\hat{f}(\mathbf{x})) = \mathbb{E}(\hat{f}_1(\mathbf{x})) = \dots = \mathbb{E}(\hat{f}_B(\mathbf{x}))$ et $V(\hat{f}(\mathbf{x})) = V(\hat{f}_1(\mathbf{x})) = \dots = V(\hat{f}_B(\mathbf{x})) = \sigma_f^2(\mathbf{x})$. On suppose que les prédicteurs sont corrélés entre eux avec un coefficient de corrélation $\rho(\mathbf{x}) \in [-1, 1]$.

D'un côté, $\mathbb{E}(\hat{f}_{bag}(\mathbf{x})) = \mathbb{E}(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})) = \mathbb{E}(\hat{f}(\mathbf{x}))$. Le bagging n'augmente donc pas le biais associé à la classe de prédicteurs. De l'autre, la variance de l'agrégat se calcule comme suit :

$$V(\hat{f}_{bag}(\mathbf{x})) = V\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})\right) \quad (3)$$

$$= \frac{1}{B^2} \left[\sum_{b=1}^B V(\hat{f}_b(\mathbf{x})) + \sum_{b \neq j} \text{Cov}(\hat{f}_b(\mathbf{x}), \hat{f}_j(\mathbf{x})) \right] \quad (4)$$

$$= \frac{1}{B^2} [B\sigma^2(\mathbf{x}) + B(B-1)\rho(\mathbf{x})\sigma^2(\mathbf{x})] \quad (5)$$

$$= \rho(\mathbf{x})\sigma^2(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{B}\sigma^2(\mathbf{x}) \quad (6)$$

Si les modèles étaient parfaitement indépendants ($\rho = 0$), la variance serait divisée par B . En pratique, puisque certaines observations sont présentes dans plusieurs sous-échantillons, le bootstrap induit une corrélation non nulle entre les modèles. Or, lorsque B devient grand, le terme $\frac{1-\rho}{B}\sigma^2$ tend vers 0. Ainsi, $V(\hat{f}_{bag}(\mathbf{x})) \approx \rho(\mathbf{x})\sigma^2(\mathbf{x}) \leq \sigma^2(\mathbf{x})$.

2.3. Erreur Out-Of-Bag (OOB)

Note

En moyenne, pour n grand, environ 63,2% des observations uniques

sont présentes dans un échantillon bootstrap. Les 36,8% restants constituent les échantillons **Out-Of-Bag** (OOB).

Preuve : La probabilité qu'un individu i ne soit pas sélectionné lors d'un tirage est $1 - 1/n$. Pour n tirages indépendants :

$$\mathbb{P}(i \notin \mathcal{D}_b) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow +\infty} e^{-1} \approx 0,368 \quad (7)$$

L'erreur OOB permet d'estimer l'erreur de généralisation sans recourir à un jeu de validation externe, en évaluant chaque observation i uniquement sur les échantillons $\mathcal{D}_b$ dont elle était absente.

3. RANDOM FOREST

3.1. Présentation

L'algorithme Random Forest améliore le bagging en cherchant à minimiser davantage la corrélation $\rho(\mathbf{x})$ entre les arbres. Pour ce faire, lors de la construction de chaque arbre, la recherche du meilleur split à chaque nœud ne se fait que sur un sous-ensemble aléatoire de $m < p$ variables. Par convention :

- Classification : $m \approx \sqrt{p}$
- Régression : $m \approx p/3$

3.2. Importance des variables

Pour un arbre CART, la variation d'impureté lors d'une division τ au nœud t est :

$$\Delta I(t, \tau) = I(t) - \left[\frac{N(t_g)}{N(t)} I(t_g) + \frac{N(t_d)}{N(t)} I(t_d) \right] \quad (8)$$

La **Diminution Moyenne de l'Impureté** (MDI) calcule l'importance d'une variable en sommant les gains d'impureté ΔI de tous les nœuds où cette variable a été choisie, pondérés par la part d'individus concernés $\frac{N(t)}{n}$, puis moyennés sur l'ensemble de la forêt. Les scores obtenus pour chaque variable sont finalement normalisés, de sorte à sommer à 1.

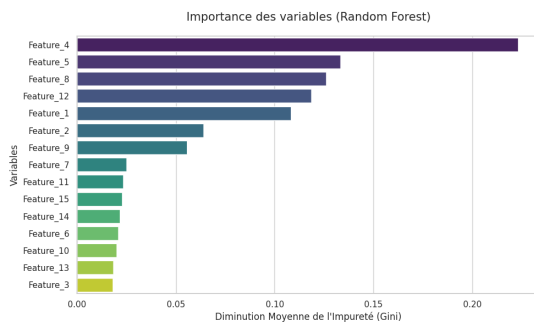


FIGURE 1. Exemple d'importance des features (MDI).

Attention : Le critère MDI est biaisé en faveur des variables à forte cardinalité (nombreuses modalités). Ces dernières ont en effet plus de chances d'être retenues pour diviser un nœud. Pour une évaluation plus robuste, l'importance par **permutation** est souvent privilégiée.

4. HYPERPARAMÈTRES CLÉS (SCIKIT-LEARN)

- `n_estimators` : Nombre d'arbres. Plus il est élevé, plus la variance théorique diminue (pas de risque de sur-apprentissage par augmentation de B). On choisit généralement entre 100 et 500 arbres.
- `max_features` : Taille du sous-ensemble aléatoire de variables. Par défaut, il vaut 'sqrt' pour classification, 'auto' = $\frac{p}{3}$ pour la régression.

- `min_samples_leaf` : Critère de pré-élagage des arbres CART. Si les nœuds enfants résultant de la division contiennent moins d'individus que `min_samples_leaf` alors la division est abandonnée, le nœud parent courant devient une feuille. Une valeur élevée réduit ainsi la variance. `min_samples_leaf` compris entre 1 et 5 est généralement un bon choix.
- `oob_score` : en spécifiant True, l'erreur OOB est calculée.

5. CONCLUSIONS ET PERSPECTIVES

Random Forest s'est imposé comme un algorithme incontournable en Data Science grâce à sa robustesse, sa faible sensibilité aux hyperparamètres et sa capacité native à gérer des relations non-linéaires et des interactions complexes. En brisant la corrélation entre les arbres par la sélection aléatoire de variables, il surmonte l'instabilité structurelle des arbres CART isolés.

Toutefois, malgré sa puissance prédictive, le Random Forest est souvent considéré comme une "boîte noire". Si le critère MDI donne une première intuition, les perspectives actuelles s'orientent vers des méthodes d'explicabilité plus fines comme les valeurs de **SHAP** (Shapley Additive Explanations). Par ailleurs, bien que les forêts aléatoires excellent pour réduire la variance, les méthodes de **Boosting** (XGBoost, LightGBM) leur sont souvent préférées pour minimiser le biais sur des jeux de données complexes, au prix d'un effort de fine tuning plus important.

CONTACT

in [linkedin.com/in/yanis-remmache-60613a151](https://www.linkedin.com/in/yanis-remmache-60613a151)

github.com/yanisrem

yanisrem.github.io